

Teoría de la Computabilidad

Módulo 1 (Repaso General): Nociones preliminares sobre lenguajes formales. Lenguajes regulares

Departamento de Cs. e Ing. de la Computación
Universidad Nacional del Sur
Bahía Blanca, Argentina

1

Material Bibliográfico

- Para una gran parte de la materia utilizaremos:
 - “Introducción a la teoría de Autómatas, Lenguajes y Computación” de John Hopcroft, Rajeev Motwani y Jeffrey Ullman, Segunda Edición (capítulos: del 1 al 9 inclusive)
 - Capítulo de 9: The Chomsky Hierarchy del libro “Introduction to Automata Theory, Languages, and Computation” de Hopcroft y Ullman (primera edición)
 - Apuntes propios de la cátedra.

2

Prof. Carlos Chesñevar / Dónde encontrarme

- Depto. de Cs. e Ing. Computación, Hall Central
- Consultas: Lunes, 11:00hs.
- E-mail: cic@cs.uns.edu.ar
- Webpage: <http://cs.uns.edu.ar/~cic>

3

¿Para qué sirve esta materia?

- En esta materia aprenderemos distintos modelos formales de qué significa realizar una “computación”.
- Muchos de estos modelos fueron desarrollados mucho antes del surgimiento de las primeras computadoras...¡pero su relevancia se mantiene!
- Los conceptos que se verán tienen repercusión directa en otras materias de las carreras del Departamento de Cs. de la Computación.

En esta clase introductoria repasaremos algunos de los conceptos que ya hemos aprendido en “Lenguajes Formales y Autómatas”



La razón: estos conceptos se vinculan directamente con los distintos temas que abordaremos en la materia.

4

Nociones preliminares

Def.: Un conjunto finito Σ es un **alfabeto** si $\Sigma \neq \emptyset$, y para todo $x \in \Sigma$, x es un símbolo indivisible. Una **palabra** o **cadena** en Σ es una **secuencia finita de símbolos de Σ**

Ejemplos:

$\Sigma = \{0,1\},$ $\Sigma = \{a,b,c,\dots,z\}$ $\Sigma = \{\#,!\}$	} son alfabetos	$\Sigma = \{00,10\}$ $\Sigma = \{>=,<,>==\}$	} NO son Alfabetos (símbolos no atómicos)
--	--------------------	---	--

Si $\Sigma = \{0,1\}$ es un alfabeto, **001** y **101** son **palabras** en Σ , pero **0011a** y **10#** no lo son.

5

Operaciones sobre cadenas

Def.: Σ^* es el conjunto de todas las cadenas formadas por elementos de Σ . Distinguimos una cadena λ (la cadena de longitud nula).

Típicamente, una palabra $w \in \Sigma^*$ se escribe $a_1 a_2 \dots a_k$, con $a_i \in \Sigma$, con $0 \leq i \leq k$. Si $k=0$ entonces se tiene la cadena nula λ (lambda)

Formalmente, decimos que $\text{long}(w)=k$, donde $\text{long}: \Sigma^* \rightarrow \text{Nat} \cup \{0\}$.

Escribimos también $|w|=k$

6

Concatenación sobre cadenas

Def.: la **concatenación de cadenas** es una operación binaria $\cdot: \Sigma^* \times \Sigma^* \rightarrow \Sigma^*$, tal que dadas cadenas $x, y \in \Sigma^*$, se define $x \cdot y = xy = w$, tal que

a) $\text{long}(w) = \text{long}(x) + \text{long}(y)$

b) $w_i = x_i$ para $i=1 \dots \text{long}(x)$, $w_{(\text{long}(x)+j)} = y_j$ para $j=1 \dots \text{long}(y)$



Potencia y reverso de cadenas

Def.: sea $w \in \Sigma^*$ una cadena. Dado un número natural i , la **potencia i -ésima de w** , notada w^i se define recursivamente como
 $w^0 = \lambda$
 $w^{i+1} = w \cdot w^i$, para todo $i \geq 0$.

Ej.: sea $\Sigma = \{0,1\}$, $w = 10$. Ent. $w^1 = 10$ y $w^2 = 1010$.

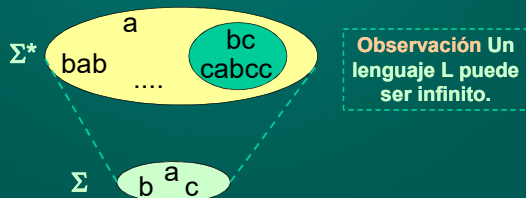
Def.: sea $w \in \Sigma^*$ una cadena, $w = a_1 a_2 \dots a_{n-1} a_n$. El **reverso** de w , notado $\bar{w}$ (o también w^r) es la cadena $w = a_n a_{n-1} \dots a_1$.
 Recursivamente:
 $\bar{\lambda} = \lambda$, si $\text{long}(w) = 0$
 $\bar{w} = a \cdot \bar{u}$, si $\text{long}(w) = n+1 > 0$, $w = u \cdot a$, $a \in \Sigma$, $u \in \Sigma^*$

Ej.: si $w = \text{pepe}$, ent. $w^r = \text{ep ep}$

Lenguajes

Def.: se denomina **lenguaje definido sobre Σ** a todo subconjunto de Σ^*

Ejemplo: sea $\Sigma = \{a,b,c\}$. Ent. $L = \{bc, cabcc\}$ es un lenguaje.



Ejemplo:

Sea $\Sigma = \{a, l, s\}$. Entonces $L = \{ala, la, las\}$ es un lenguaje definido sobre Σ .

Lenguajes

Dado que un lenguaje es un conjunto, podemos considerar las operaciones habituales de **unión** ($L_1 \cup L_2$), **intersección** ($L_1 \cap L_2$), **diferencia** ($L_1 - L_2$) y **complemento**.

Otras operaciones

- Concatenación de lenguajes
- Potencia de lenguajes
- Estrella de Kleene

Concatenación

Sean L_1, L_2 dos **lenguajes** definidos sobre Σ . Entonces su **concatenación L** se define como

$$L = L_1 \cdot L_2 = \{w | w = x \cdot y, \text{ para algún } x \in L_1, y \in L_2\}$$

Ejemplo:

Sea $\Sigma = \{a..z\}$. Entonces

$$L_1 = \{ja, je\}$$

$$L_2 = \{jo, ju\}$$

son **lenguajes definidos a partir de Σ** .

$$\text{Sea } L = L_1 \cdot L_2 = \{jajo, jaju, jejo, jeju\}$$

Potencia

Sea $L \subseteq \Sigma^*$. Definiremos la **potencia de lenguajes** como

$$L^0 = \{\lambda\}$$

$$L^{i+1} = L \cdot L^i, \text{ para todo } i \geq 0.$$

Ejemplo:

Sea $\Sigma = \{a..z\}$. Entonces $L = \{ja, je\}$ es un **lenguaje definido a partir de Σ** .

La potencia L^3 contiene, entre otros, a
 jajaja, jajaje, jejaje

Estrella

Def.: sea L un lenguaje. La **estrella** (o **clausura**) de **Kleene**, notada L^* , se define como

$$L^* = \bigcup_{i=0}^{\infty} L^i$$

Definimos también $L^+ = \bigcup_{i=1}^{\infty} L^i$, para $i=1 \dots \infty = L \cdot L^*$

$$\Sigma^* = \bigcup_{i=1}^{\infty} \Sigma^i, \Sigma^+ = \Sigma \cdot \Sigma^* = \Sigma^* - \{\lambda\}$$

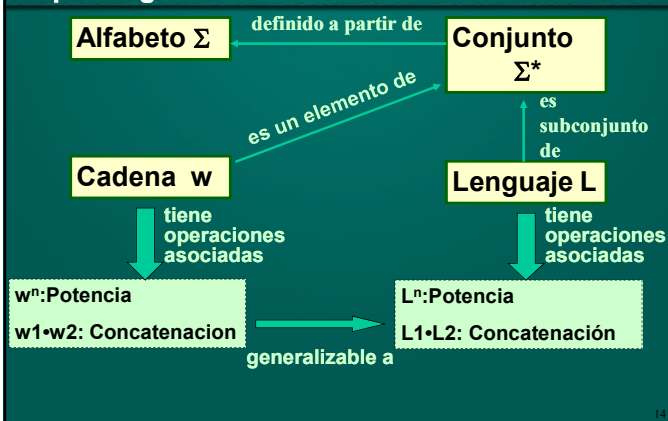
Ejemplo:

Sea $\Sigma = \{a..z\}$. Entonces $L = \{ja, je\}$ es un **lenguaje definido a partir de Σ** .

La cadena **jajejeje** pertenece al conjunto L^4 , y por lo tanto pertenece a L^*

13

Esquema general



14

Autómatas Finitos (AF)

Un **autómata finito** es una máquina o ente abstracto que se estudiará desde dos puntos de vista:

- como **traductor** de una cadena a otra.
- como **dispositivo reconocedor** de que una cadena pertenece a un lenguaje determinado

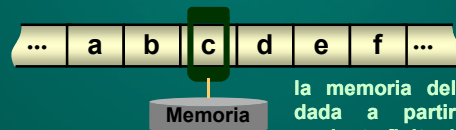
Un AF puede imaginarse como una máquina capaz de leer símbolos de una cinta.

Esta “máquina” posee diferentes **estados**, y puede pasar de un estado a otro dependiendo de lo que lee en la cinta...

15

Autómatas Finitos (AF)

Los AF que veremos pueden **leer** símbolos de una cinta y **reaccionar** ante ellos.



la memoria del AF está dada a partir de un conjunto finito de estados

Estados del AF

- estado **inicial**
- estado **final o aceptador**
- estados **intermedios**

Dada una cadena w en la cinta, si el AF **termina** [no termina] en un estado final, decimos que la cadena es **aceptada** [rechazada] por el autómata.

16

Autómatas Finitos Reconocedores (AFR)

Definición: un **autómata finito reconocedor** (AFR) es una quintupla

$$M = (S, \Sigma, \delta, s_0, F),$$

donde:

S es un **conjunto finito de estados**, $S \neq \emptyset$

Σ es el **alfabeto de entrada**

$\delta: S \times \Sigma \rightarrow S$ es la **función de transición**

s_0 es el **estado inicial**, $s_0 \in S$.

F es el **conjunto de estados finales o aceptadores** ($F \subseteq S$)

17

AFR: Ejemplo

Considérese el lenguaje L de los identificadores válidos del lenguaje Pascal, sin tener en cuenta la restricción sobre su longitud máxima. Sabemos que:

- los identificadores comienzan siempre por una letra
- luego pueden continuar con letras o dígitos

¿Cómo podemos diseñar un AF que sea capaz de reconocer este lenguaje?

18

AFR: Ejemplo (cont.)

Podemos diseñar el siguiente autómata finito reconocedor

$M_{\text{ident}} = (S, \Sigma, \delta, s_0, F)$, con

$S = \{s_0, s_1, s_2\}$

$\Sigma = \{a..z, 0..9\}$

$F = \{s_1\}$

δ = definida de acuerdo a la siguiente tabla.

δ	letra	dígito
$\rightarrow s_0$	s_1	s_2
$* s_1$	s_1	s_1
s_2	s_2	s_2

s_1 = estado aceptador

s_2 = estado rechazador

IMPORTANTE: La función δ debe estar completamente definida

19

Convenciones

Dado un autómata

$M = (S, \Sigma, \delta, s_0, F)$,
podemos representarlo a través de un digrafo
 $G = (N, A, V)$
usando las siguientes convenciones:

- Existe un nodo $n \in N$ en G por cada estado $s \in S$
- Existe un arco $a \in A$ tal que $V(a) = (s_i, s_k)$ sssi $\delta(s_i, a) = s_k$, donde $s_i, s_k \in S$, $a \in \Sigma$.
- El Estado inicial se indica con " $\rightarrow$ "; los estados finales con **doble círculo**.

20

Ejemplo de grafo

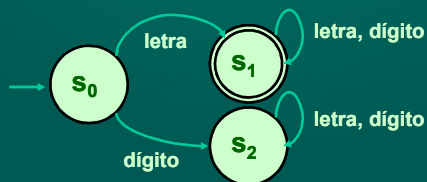
$M_{\text{ident}} = (S, \Sigma, \delta, s_0, F)$, con

$S = \{s_0, s_1, s_2\}$;

$\Sigma = \{a..z, 0..9\}$; $F = \{s_1\}$

δ = definida por la tabla.

δ	letra	dígito
$\rightarrow s_0$	s_1	s_2
$* s_1$	s_1	s_1
s_2	s_2	s_2



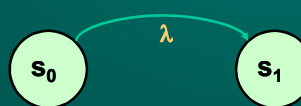
21

Extensiones de AFs - Transiciones λ

Hasta ahora el AF realizaba un cambio de estado únicamente al consumir un símbolo en la cinta.

Una extensión de los AF nos permite realizar cambios de estado sin la necesidad de leer (consumir) símbolos.

Las transiciones que no consumen símbolos de la cinta se denominan **λ -transiciones**.



¡Este agregado no incrementa la potencia de los AF!

Lema: Dado un autómata finito M que permite transiciones etiquetadas con λ , puede construirse otro autómata M' equivalente sin transiciones etiquetadas con λ .

22

AFR: definición revisada

Def.: un **autómata finito reconocedor** (AFR) es una quintupla $M = (S, \Sigma, \delta, s_0, F)$, donde

S es un **conjunto finito de estados**, $S \neq \emptyset$

Σ es el **alfabeto de entrada**

$\delta: S \times (\Sigma \cup \{\lambda\}) \rightarrow S$ es la **función de transición**

s_0 es el **estado inicial**, $s_0 \in S$.

F es el **conjunto de estados finales** o **aceptadores** ($F \subseteq S$)

23

Configuración. Transición

Def.: Sea $M = (S, \Sigma, \delta, s_0, F)$, llamaremos **configuración** a cada par $(s, w) \in S \times \Sigma^*$.

Una configuración es un par estado-cadena que representa la situación global en la que se encuentra M en un momento dado.

(s, w) puede interpretarse como una "foto" del autómata en cuestión que nos indica:

"El autómata está en el estado s , y falta consumir la cadena w de la cinta".



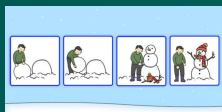
24

Configuración. Transición

Def.: Una **transición** de M será representada por una relación binaria " $\vdash$ " entre configuraciones: $(s, aw) \vdash (s', w)$ sssi $\delta(s, a) = s'$, donde $s, s' \in S$, $a \in \Sigma \cup \{\lambda\}$; $w \in \Sigma^*$.

Una transición indica la evolución del autómata.

Informalmente, estamos pasando de una "foto" a otra "foto". **Notación:** $\vdash^*$ y $\vdash^+$



$(s, aw) \xrightarrow{\delta(s, a)=s'} (s', w)$

25

Reconocimiento y aceptación

Def.: El AF $M = (S, \Sigma, \delta, s_0, F)$ **acepta** una cadena $w \in \Sigma^*$ sssi $(s_0, w) \vdash^* (s_f, \lambda)$, con $s_f \in F$.

Def.: Sea un AFR $M = (S, \Sigma, \delta, s_0, F)$, el lenguaje **reconocido** por el autómata M es $L(M) = \{w \in \Sigma^* \mid w \text{ es aceptada por } M\}$.

Ejemplo Identificadores de Pascal:

$(s_0, \text{cant}) \vdash (s_1, \text{ant}) \vdash^+ (s_1, \lambda)$, entonces podemos decir que el M_{ident} **acepta cant**.

Sin embargo, $(s_0, \text{7cant}) \vdash (s_2, \text{cant}) \vdash^+ (s_2, \lambda)$. Luego M_{ident} **no acepta 7cant**.

26

AF no deterministas

Def.: Un **AFND** (AF no determinista) es una quintupla

$$M = (S, \Sigma, \delta, s_0, F)$$

donde

S es un conjunto finito de estados, $S \neq \emptyset$

Σ es el alfabeto de entrada

$\delta: S \times (\Sigma \cup \{\lambda\}) \rightarrow P(S)$ (conjunto de partes de S) es la función de transición

s_0 es el estado inicial, $s_0 \in S$.

F es el conjunto de estados finales ($F \subseteq S$)

Ahora tenemos $\delta(s_0, a) = \{s_1, s_{i+1}, \dots, s_m\}$. El próximo estado será un elemento a elegir de este conjunto.

27

Solución con AFND

Debemos adaptar la definición de *cadena aceptada por el AFND...*

Dado que tenemos varios caminos posibles para una misma cadena, ésta será *aceptada* si con ella podemos alcanzar un estado aceptador en el AFND.

Def.: Se dice que una cadena w es **aceptada** por un **AFND** $M = (S, \Sigma, \delta, s_0, F)$ si existe $p \in F$ tal que $\delta^*(s_0, w) \vdash^* (p, \lambda)$.

Notar que la aceptación no depende de la "elección" del camino: si éste existe, w será aceptada...

28

Determinismo vs. No determinismo

"Determinismo" es un término de gran importancia en muchas áreas de Ciencias de la Computación.

Determinismo: no hay elección posible. O bien existe un alternativa válida, o no hay alternativa.

No Determinismo: hay varias alternativas válidas.

Al utilizar un formalismo para realizar computaciones, es importante distinguir si tener **no determinismo** brinda mayor potencia de cómputo.

En el caso de los automatas finitos, el no determinismo **no** incrementa su poder de cómputo.

29

Expresiones Regulares. Introducción

Para describir un lenguaje tenemos entonces:

- La descripción conjuntista del lenguaje.
- La descripción procedimental del lenguaje (dada a través del autómata finito)
- La descripción algebraica del lenguaje (dada a través de una expresión regular)

Además, sería ideal que exista un procedimiento para obtener la representación algebraica (expresión regular) de todas las cadenas a partir de un AF que las reconoce.

30

Lenguajes regulares (LR)

Los lenguajes que podremos denotar por medio de ER los llamaremos **lenguajes regulares**

Def.: los lenguajes regulares (LR) son la clase de los lenguajes representables por ER. Definimos una función $L : A \rightarrow \Sigma^*$, tal que si α es una ER ent. $L(\alpha)$ es el lenguaje representado por α .

Def.: Función $L : A \rightarrow \Sigma^*$,

- 1) $L(\lambda) = \{\lambda\}$.
- 2) $L(\emptyset) = \emptyset$.
- 3) Para cada $a \in \Sigma$, $L(a) = \{a\}$.
- 4) Si α, β son ER, ent. $L(\alpha^* \beta) = L(\alpha) \cdot L(\beta)$
- 5) Si α, β son ER, ent. $L(\alpha + \beta) = L(\alpha) \cup L(\beta)$
- 6) Si α es una ER, ent. $L(\alpha^*) = L(\alpha)^*$

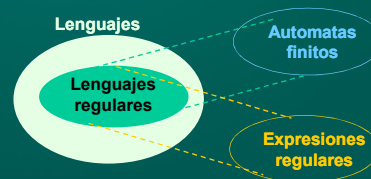
31

Lenguajes Regulares y Automatas finitos

Los lenguajes regulares nos interesan porque son exactamente los lenguajes que reconoce un autómata finito.

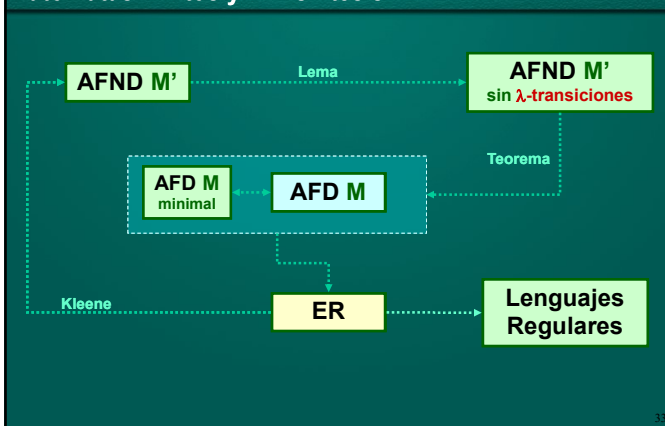
Esta correspondencia es establecida por Kleene en un teorema que relaciona los lenguajes que pueden ser descriptos con una **expresión regular** con los lenguajes que puede reconocer un autómata finito.

Teorema de Kleene: un lenguaje L definido por una expresión regular puede definirse también mediante un AF.



32

Autómatas Finitos y ER: síntesis



33

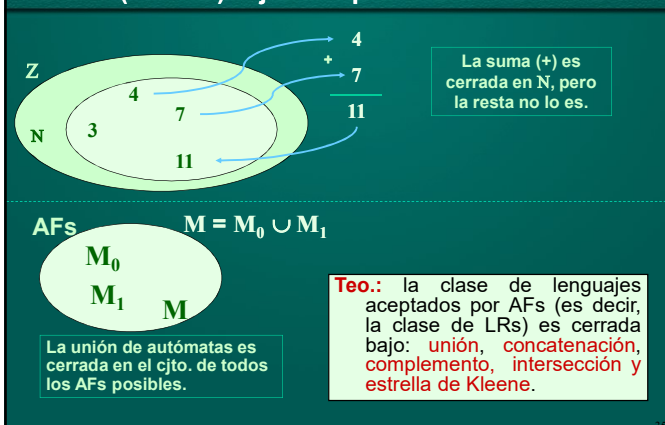
Propiedades de los LR

Existen propiedades de la clase de lenguajes aceptados por AF obtenidas por **clausura bajo ciertas operaciones**. Estas propiedades son importantes, pues:

- brindan herramientas para construir y simular AFs
- esclarecen el nexo entre AFs y ERs
- muestran que los LR son *estables*
- ayudan a identificar el tipo de un lenguaje
- simplifican la demostración de resultados teóricos

34

Clausura (Closure) bajo una operación



35

Otras propiedades

Los lenguajes regulares son además cerrados bajo las siguientes propiedades:

- **diferencia** ($L - M = L \text{ intersectado } M \text{ complemento}$)
- **reflexión** (es el lenguaje reverso)
 1. Invertir los arcos del autómata.
 2. Estado inicial original = único estado aceptador
 3. λ -transiciones de un estado inicial nuevo a los aceptadores originales
- **homomorfismo** (sustitución de símbolos por cadenas)
- **homomorfismo inverso** (deshacer un homomorfismo)

36

Gramáticas estructuradas por frases

Def.: Una **gramática estructurada por frases (GEF)** es una 4-upla

$$G=(V_n, V_t, S, P)$$

donde

V_n = conj. finito de **símbolos no terminales** (o auxiliares)

V_t = conj. finito de **símbolos terminales**

S = **símbolo inicial**

P = conj. finito de **reglas de producción** $\alpha \rightarrow \beta$, donde

$$\alpha \in (V_n \cup V_t)^* V_n (V_n \cup V_t)^*, \text{ y}$$

$$\beta \in (V_n \cup V_t)^*.$$

Las reglas de producción (o reescritura) permiten generar **cadenas** o **palabras**.

37

Gramáticas estructuradas por frases

De ahora en adelante, en toda GEF asumiremos

$$V_t \neq \emptyset \quad \text{y} \quad V_t \cap V_n = \emptyset.$$

Una regla de producción especial será $\alpha \rightarrow \lambda$ (**regla de borrado**).

Ejemplos:

Consideremos la gramática $G=(V_n, V_t, S, P)$, con $V_n = \{S\}$,
 $V_t = \{0, 1\}$, $P = \{S \rightarrow 0S, S \rightarrow 1\}$

¿Qué lenguaje genera esta gramática?

• Pregunta típica: dada una gramática, saber qué lenguaje es generado por ella.

38

La noción de derivación

$G=(V_n, V_t, S, P)$,
 $V_n = \{S\}$,
 $V_t = \{0, 1\}$,
 $P = \{S \rightarrow 0S, S \rightarrow 1\}$

Recordemos que un lenguaje es un conjunto de cadenas. Veamos qué cadenas podemos producir con esta gramática G

Aplicando la regla	Obtenemos
$S \rightarrow 1$	1
$S \rightarrow 0S$	0S
$S \rightarrow 1$	01
$S \rightarrow 0S$	0S
$S \rightarrow 0S$	00S
$S \rightarrow 1$	001

Intuitivamente,
el lenguaje
resultante es
 0^*1

39

Derivación : noción

Cada paso de los anteriores se llama **derivación**. Dada una gramática G, dos cadenas α_1 y α_2 están relacionadas a través de una derivación si existen reglas de producción para reescribir una parte de α_1 en otra parte de α_2 . Distinguimos:

Derivación directa ($\xrightarrow{G}$): la cadena resultante se obtiene en un paso usando las reglas de G.

Derivación (general) ($\xrightarrow{G^+}$): la cadena resultante se obtiene en uno o más pasos usando las reglas de G.

40

Derivación directa (def. formal)

Def.: Sea $G=(V_n, V_t, S, P)$ una gramática, y $\mu, \varphi \in (V_n \cup V_t)^*$, se dirá que $\mu\alpha\varphi$ **deriva directamente a** $\mu\beta\varphi$ en G (denotado como $\mu\alpha\varphi \xrightarrow{G} \mu\beta\varphi$) si existe una regla de producción $\alpha \rightarrow \beta \in P$.

Ejemplo:

Consideremos la gramática $G=(V_n, V_t, S, P)$, con $V_n = \{S\}$, $V_t = \{0, 1\}$, $P = \{S \rightarrow 0S, S \rightarrow 1\}$.

Algunas derivaciones directas son:

$$S \xrightarrow{G} 0S, \quad 0S \xrightarrow{G} 00S, \quad 00S \xrightarrow{G} 001$$

41

Derivación (def. formal)

Def.: Dadas dos cadenas α, β , diremos que α **deriva a** β según la gramática G, notado $\alpha \xrightarrow{G^+} \beta$, si β puede obtenerse por aplicación de reglas de producción de G a partir de α , esto es

$$\alpha \xrightarrow{G} \gamma_1, \quad \gamma_1 \xrightarrow{G} \gamma_2, \dots, \gamma_{j-1} \xrightarrow{G} \gamma_j \xrightarrow{G} \beta$$

Def.: Sea $G=(V_n, V_t, S, P)$ una gramática. Entonces: $L(G) = \{w \in V_t^* \mid S \xrightarrow{G^+} w\}$ es el **lenguaje generado** por G

Ejemplo: Sea $G=(V_n, V_t, S, P)$, con $V_n = \{S\}$, $V_t = \{0, 1\}$, $P = \{S \rightarrow 0S, S \rightarrow 1\}$.

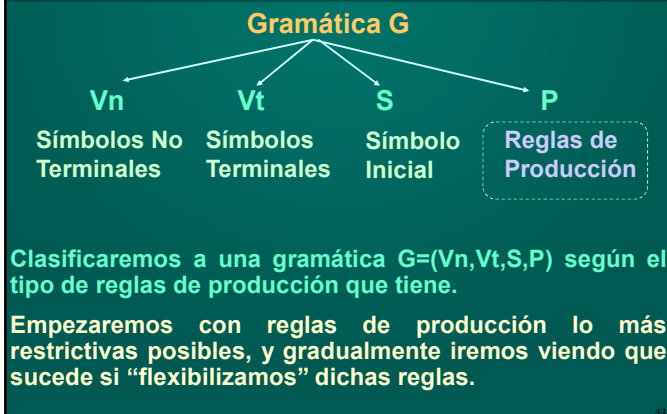
Entonces

$$S \xrightarrow{G^+} 000001, \quad S \xrightarrow{G^+} 00000000000001$$

y resulta $L(G) = \{1, 01, 001, 0001, \dots\} = L(0^*1)$

42

Clasificando Gramáticas

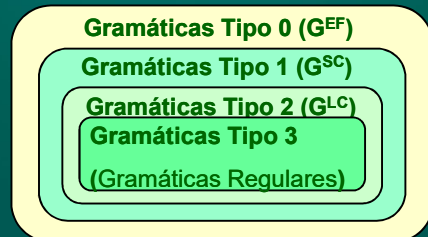


Clasificación de gramáticas de Chomsky

Entre 1950 y 1960, el lingüista norteamericano **Noam Chomsky** desarrolló una clasificación de las gramáticas. Su idea: modelar los lenguajes naturales, con miras a la traducción automática



Noam Chomsky



Gramáticas Regulares

Def.: Una **gramática regular (GR)** es una 4-upla $G=(V_n, V_t, S, P)$, donde

V_n = conj. finito de **símbolos no terminales**

V_t = conj. finito de **símbolos terminales**

S = **símbolo inicial**

P = conj. finito de **reglas de producción** $\alpha \rightarrow \beta$ t. que:

$\alpha \in V_n$ (es un único símbolo no-terminal)

β es un único terminal ó es un terminal concatenado con un no-terminal, es decir $\beta \in V_t$ ó $\beta \in V_t V_n$

Observación:

si $\beta \in V_t V_n$, se habla de una GR recursiva a derecha.

si $\beta \in V_n V_t$, se habla de una GR recursiva a izquierda.

Gramáticas Regulares: Ejemplos

Diseñar una gramática para generar las cadenas de $\{a,b\}^+$ que empiezan con a: $a(a+b)^*$.

$P=\{S \rightarrow a, \quad S \rightarrow aB, \quad B \rightarrow bB, \quad B \rightarrow aB, \quad B \rightarrow a, \quad B \rightarrow b\}$

$G=(V_n, V_t, S, P),$
 $V_n=\{S, B\},$
 $V_t=\{a, b\},$

¿abbaba $\in L(G)$?

$S \xrightarrow{G} aB \xrightarrow{G} abB \xrightarrow{G} abbB \xrightarrow{G} abbaB \xrightarrow{G} abbabB \xrightarrow{G} abbaba$

Como $S \xrightarrow{G^+} abbaba$, resulta **abbaba** $\in L(G)$

Gramáticas Regulares: Ejemplos

Diseñar una GR para generar las cadenas de $\{a,b\}^+$ que contienen 3 b's consecutivas, $(a+b)^*bbb(a+b)^*$.

$P=\{S \rightarrow aS, \quad S \rightarrow bS, \quad S \rightarrow bT_1, \quad T_1 \rightarrow bT_2, \quad T_2 \rightarrow bT_3, \quad T_3 \rightarrow b, \quad T_3 \rightarrow aT_3, \quad T_3 \rightarrow bT_3, \quad T_3 \rightarrow a, \quad T_3 \rightarrow b\}$

$G=(V_n, V_t, S, P),$
 $V_n=\{S, T_1, T_2, T_3\},$
 $V_t=\{a, b\},$

¿abbabbb $\in L(G)$?

$S \xrightarrow{G} aS \xrightarrow{G} abS \xrightarrow{G} abbS \xrightarrow{G} abbaS \xrightarrow{G} abbabT_1 \xrightarrow{G} abbabbT_2 \xrightarrow{G} abbabbb$

Como $S \xrightarrow{G^+} abbabbb$, resulta **abbabbb** $\in L(G)$

Gramáticas Regulares: Ejemplos

Diseñar una GR para generar el lenguaje de identificadores de Pascal (cualquier longitud).

$G=(V_n, V_t, S, P),$
 $V_n=\{S_0, S_1\},$
 $V_t=\{a...z\} \cup \{A...Z\} \cup \{0...9\}$

$P=\{S_0 \rightarrow a | b | c | \dots | z, \quad S_0 \rightarrow A | B | C | \dots | Z, \quad S_0 \rightarrow aS_1 | bS_1 | cS_1 | \dots | zS_1, \quad S_0 \rightarrow AS_1 | BS_1 | CS_1 | \dots | ZS_1, \quad S_1 \rightarrow aS_1 | bS_1 | cS_1 | \dots | zS_1, \quad S_1 \rightarrow AS_1 | BS_1 | CS_1 | \dots | ZS_1, \quad S_1 \rightarrow 0S_1 | 1S_1 | 2S_1 | \dots | 9S_1, \quad S_1 \rightarrow 0 | 1 | 2 | \dots | 9, \quad S_1 \rightarrow a | b | c | \dots | z, \quad S_1 \rightarrow A | B | C | \dots | Z\}$

Equivalencias entre GRs y AFs

Veremos que dos formalismos diferentes en apariencia son similares en su poder expresivo.

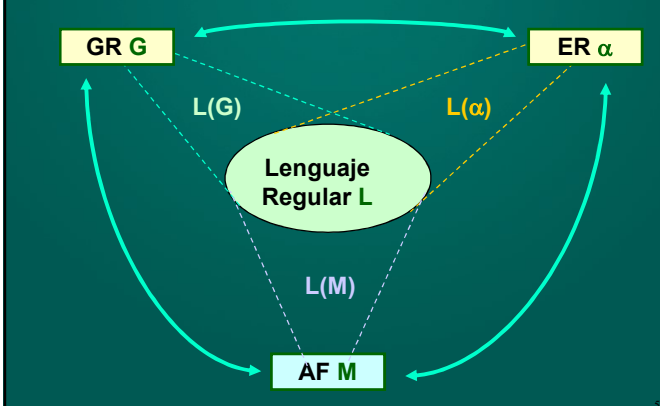
Teo.: sea G una GR. Entonces

$L(G)$ es generado por $G \leftrightarrow \exists \text{ AF } M$, tal que $L(M) = L(G)$.

Este teorema establece una **equivalencia**: existe un AF para el lenguaje generado por una gramática G , y existe una GR para el lenguaje reconocido por un AF M

49

Triple Equivalencia



50

Pruebas sobre Lenguajes Regulares

Pruebas
sobre
Lenguajes

Probar que un
lenguaje **es** regular

Probar que un
lenguaje **no es** regular

51

Probar que un Lenguaje es Regular

Si un
lenguaje es
regular,
sabemos
que

Puede especificarse con una **ER**.

Puede asociarse a él un **AF**
reconocedor.

Es **cerrado** bajo unión, concat.,
estrella de Kleene, complemento e
intersección

Es generable por una **gramática**
regular

Estos hechos pueden ser utilizados en forma individual o combinados para demostrar que un lenguaje es regular

52

Probar que un Lenguaje es Regular

Ejemplo: $\Sigma = \{a,b\}$, $L \subseteq \Sigma^*$ el conjunto de todas las cadenas de longitud impar que contienen un número par de letras a , probar que L es regular.

Podemos probar que L es regular si L está definido por la intersección de dos lenguajes regulares.

L_1 = cadenas de long. impar = $(a+b)((a+b)(a+b))^*$

L_2 = cadenas con cant. par de a 's: = $b^*(ab^*ab^*)^*$

Notemos que $L = L_1 \cap L_2$. **Luego L es regular.**

53

Probar que un Lenguaje es Regular

Notemos que si L es un lenguaje finito, entonces es un lenguaje regular.

Ejemplo: El lenguaje formado por cadenas de hasta 2 a 'es

$$L = \{a^n \mid n < 3\}$$

Este lenguaje finito es regular. Lo mismo vale para

$$L = \{a^n b^m \mid n < 100, m < 50\}.$$

Pero existen también lenguajes regulares que no son finitos. Mas aún, no todos los lenguajes infinitos son regulares...

El lenguaje $L = \{a^n b^m c^p \mid n, m, p \geq 0\}$, es un lenguaje regular infinito.

El lenguaje $L = \{a^n b^n \mid n > 0\}$ es infinito. ¿Es regular?

54

Probar que un Lenguaje es Regular

Para probar que un lenguaje infinito **es** regular, podemos

- buscar la **gramática regular** que lo genera, o
- el **autómata finito** que lo reconoce, o
- la **expresión regular** que describe la forma de sus cadenas, o
- utilizar las **propiedades de clausura**, tomando lenguajes que sabemos (o debemos probar) son regulares.

Obviamente, que no encontremos estos elementos, no permite probar que el lenguaje no es regular!

Es como decir que en Bahía Blanca no hay ninguna Ferrari pues hasta ahora no hemos visto ninguna...

55

Probar que un Lenguaje NO es Regular

Para afirmar que un lenguaje **L no es** regular necesitamos una propiedad que caracterice a los LR y que sea “fácil” demostrar que no se cumple en L

Una propiedad importante para los lenguajes infinitos la indica el siguiente teorema:

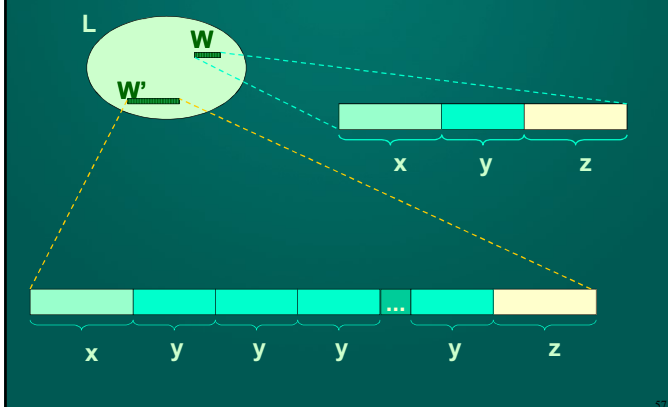
Teo.: si L es un lenguaje **regular** (infinito). Entonces **existe** una constante k (que depende de L), tal que **para toda** cadena **w** $\in L$, con $\text{long}(w) \geq k$, podemos dividir **w** en tres cadenas, **w=xyz**, de modo que:

1. **y** $\neq \lambda$
2. $|xy| \leq k$
3. **para todo** $n \geq 0$, la cadena **xyⁿz** $\in L$

Teorema de la familia de los “Pumping theorems”

56

Probar que un Lenguaje es NO Regular



57

Pumping theorem para lenguajes regulares

Dem.[Esquema]:

Podemos entender el enunciado del teorema apelando a la relación **AF – LR**.

Supongamos que L es regular y que M es el AF que reconoce cadenas de L. Supongamos que M tiene **n estados**.

Consideremos una cadena **w** de longitud **n** o más:

$$w = a_1 a_2 a_3 \dots a_m \quad \text{donde } m \geq n$$

Definimos cada estado **s_i** como $\delta(s_0, a_1 a_2 \dots a_i)$. Esto es, **s_i** es el estado en el que está el AF M después de consumir los primeros **i** símbolos de la cadena

58

Probar que un Lenguaje es NO Regular

Como hay más símbolos para leer que estados para recorrer, sin duda repetiremos algún estado más de una vez...

Entonces si definimos cada estado **s_i** como $\delta(s_0, a_1 a_2 \dots a_i)$ podemos encontrar dos enteros **i** y **j**, $0 \leq i < j \leq n$, tal que **s_i = s_j**. Podemos dividir **w** en **x, y, z** tal que:

$$x = a_1 a_2 \dots a_i$$

La cadena **x** nos lleva al estado **s_i**.

$$y = a_{i+1} a_{i+2} \dots a_j$$

De alguna manera volvemos al mismo estado luego de procesar **y**, que nos lleva al estado **s_j = s_i**

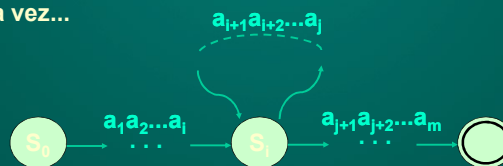
$$z = a_{j+1} a_{j+2} \dots a_m$$

Finalmente, avanzamos hacia el estado aceptador con **z**

59

Probar que un Lenguaje es NO Regular

Como hay más símbolos para leer que estados para recorrer, sin duda repetiremos algún estado más de una vez...



Si tomamos **x = a₁a₂...a_i**, **y = a_{i+1}a_{i+2}...a_j**, **z = a_{j+1}a_{j+2}...a_m**
Luego **xyⁿz** también pertenece al lenguaje L.

60

Teo. de Pumping para Leng.Regulares

Es un teorema delicado...Tenemos varios niveles

Para todo lenguaje regular L (infinito)

existe $k \geq 1$, tal que

para toda cadena $w \in L$ con $\text{long}(w) \geq k$,

existen $x, y, z \mid w = xyz$, $\text{long}(xy) \leq k$, $y \neq \lambda$ tal que

para todo $n \geq 0$, $xy^n z \in L$

61

Ejemplo: $L = \{a^p b^p \mid p \geq 0\}$ no es regular

Probemos que $L = L = \{a^p b^p \mid p > 0\}$ no es un lenguaje regular.



RECORDAR

El teorema no puede utilizarse para demostrar que un leng. **es regular**. Solo puede utilizarse en demostraciones por el absurdo para garantizar que **no lo es**.

La condición dada por el teorema conforman una condición necesaria pero no suficiente.

62

Ejemplo: $L = \{a^p b^p \mid p \geq 0\}$ no es regular

Ej: probemos que $L = L = \{a^p b^p \mid p > 0\}$ es no regular.



Dem: supongamos por el absurdo que L es regular. Ent. según teo. pumping existe un k , tal que para toda $w \in L$ con $\text{long}(w) \geq k$, $w = xyz$ con $y \neq \lambda$, $\text{long}(xy) \leq k$ entonces $xy^n z \in L$, para todo $n \geq 0$.

Sea $w = a^k b^k \in L$, $\text{long}(w) = 2k \geq k$

Como veremos a continuación, hay un solo caso posible de particionar $w = xyz$ con $\text{long}(xy) \leq k$

63

i) y está formada solo por **a's**:

$x = a^p$, donde: $k = p + q + r$
 $y = a^q$, $\text{long}(xy) \leq k \Rightarrow p + q \leq k$
 $z = a^r b^k$, $y \neq \lambda \Rightarrow q > 0$.

Luego $xy^n z = a^{p+nq+r} b^k = a^{p+nq+r} b^k$, para $n > 0$.

Para que $a^{p+nq+r} b^k \in L$ debe tener igual cantidad de **a's** y **b's**, es decir $k = p + nq + r$. Pero... ¡eso sólo pasa si $n=1$! Luego $xy^n z \notin L$, para $n > 1$.

64

ii) y está formada solo por **b's**:

$x = a^k b^p$, donde: $k = p + q + r$
 $y = b^q$, $y \neq \lambda \Rightarrow q > 0$.
 $z = b^r$,

Note que $xy = a^k b^{p+q}$ y como $q > 0$ luego $\text{long}(xy) = k + p + q > k$ y por lo tanto no respeta la condiciones del teorema.

iii) y está formada solo por **a's** y **b's**:

$x = a^p$, donde: $k = p + q + r + s$
 $y = a^q b^r$, $y \neq \lambda \Rightarrow q + r > 0$.
 $z = b^s$

Note que $xy = a^{p+q} b^r = a^k b^r$, luego $\text{long}(xy) = k + r > k$ y por lo tanto no respeta la condiciones del teorema.

65

Conclusión: No hay forma de descomponer $w = xyz$, con $\text{long}(xy) \leq k$ tal que podamos asegurar que $xy^n z \in L$, para todo $n > 0$. Esto partió de suponer que L era regular. Luego, ¡ L no es regular! CQD.

66

Aplicaciones

Compiladores de Lenguajes
de Programación

Front-Ends de Bases de
Datos

Procesamiento de lenguaje
natural

Formalizables a través
de Autómatas Finitos



Gramáticas Regulares



Expresiones Regulares